

ICML 2022 · Telling a Robot What to Do by Analogy

Berkeley and Meta AI propose goal-conditioned bisimulation — a representation where analogous tasks line up, and goals can be composed by arithmetic.

Most goal-conditioned reinforcement learning starts from a convenient assumption: that you can hand the agent the exact state it is supposed to reach, usually as a target image. Give the policy a picture of the closed drawer, and it learns to make the world look like that picture. It is a clean formulation, and it works well in the lab.

The trouble is that you rarely have that picture. If you ask a robot to close a drawer it has never seen, you do not know in advance what “closed” looks like for that particular drawer — its color, its position, the clutter around it. The exact goal state is precisely the thing you cannot specify. What you can usually provide is an example: here is a different drawer being closed; now do the same to yours.

This ICML 2022 paper from UC Berkeley and Meta AI takes that idea seriously. It proposes goal-conditioned bisimulation (GCB), a way of learning representations in which analogous tasks — closing a red drawer, closing a blue one — land in the same place. Once tasks line up like this, a goal can be described not by an image but by analogy, and recovered with simple arithmetic in latent space.

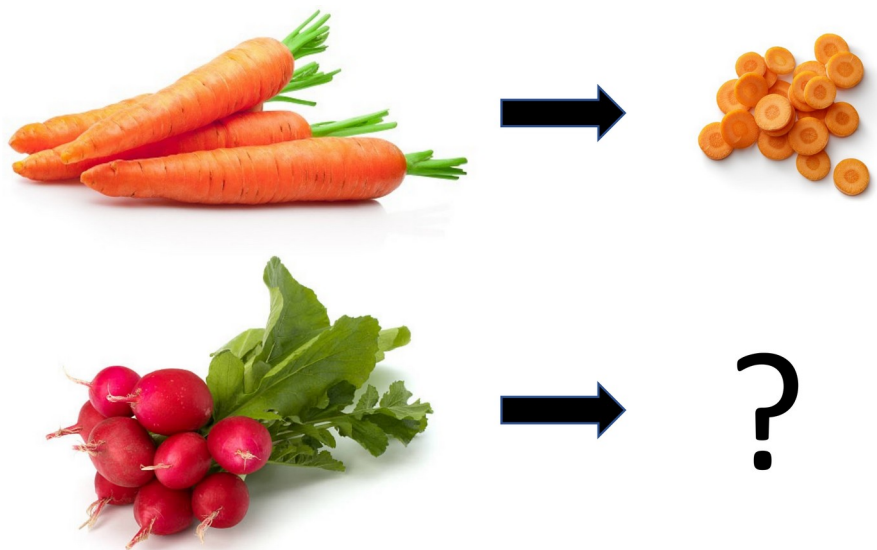


Figure 1. The guiding analogy. Dicing a carrot and dicing a radish involve different objects, but the same skill and the same functional change from start to goal. GCB aims to use this kind of analogy to specify goals in new, unseen environments.

The vegetable example is the paper's north star. A person who can dice a carrot can dice a radish without being shown the exact final pile of radish slices; they carry a representation of the task that ignores what is irrelevant (which vegetable) and preserves what matters (the cut). The paper's question is how to learn such a representation for an RL agent directly from pixels.

From equivalent states to equivalent tasks

GCB builds on bisimulation, a classical idea in RL for grouping states. Two states are bisimilar if they are indistinguishable in terms of the rewards they produce under every possible action sequence; bisimulation metrics soften this into a distance that measures how behaviorally similar two states are. Recent representation-learning work used these metrics to make agents robust to visual distractors — but always for a single task.

The paper's move is to change what is being grouped. Instead of grouping equivalent states, GCB groups equivalent tasks — that is, state-goal pairs that behave the same way. Two pairs (s, g) are treated as equivalent when they demand the same actions and lead to the same distribution over future goals. The authors call the property this captures functional equivariance: the representation stays invariant to details that do not matter (color, background objects) and changes predictably with the details that do (which drawer, pressed or not).

The method: two encoders that make goals composable

To turn that principle into something learnable, GCB trains two encoders together. A state-goal encoder ϕ maps a pair (s, g) to a vector, and is trained so that the ℓ_1 distance between two encodings matches an on-policy goal-conditioned bisimulation metric — the behavioral distance between the two tasks. A separate state encoder ψ maps a single state, and is trained so that $\psi(g) - \psi(s)$ equals $\phi(s, g)$. That second constraint is the crucial one: it forces the task representation to behave like a difference of state representations, which is exactly what makes goals composable.

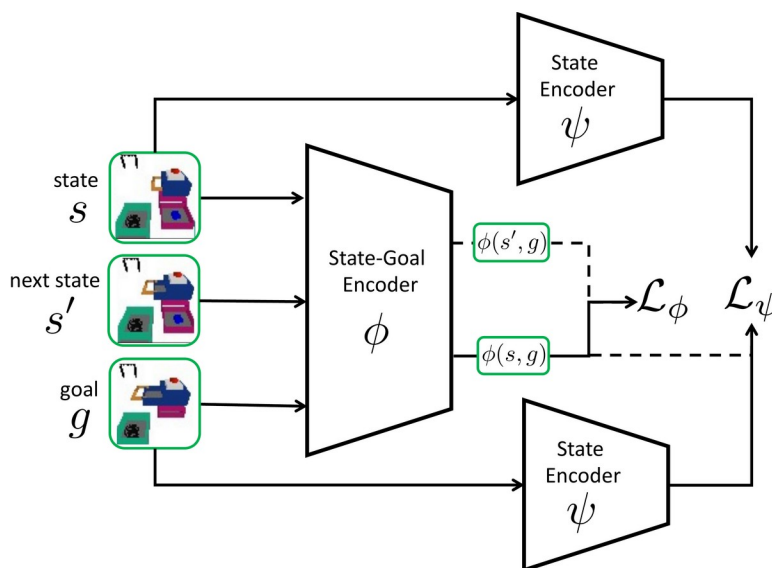


Figure 3. The representation-learning core of GCB. The state-goal encoder ϕ is fit to the bisimulation metric (loss $\mathcal{L}_\phi$), while the shared-weight state encoder ψ is trained so that $\psi(g) - \psi(s)$ matches $\phi(s, g)$ (loss $\mathcal{L}_\psi$). Dashed lines are stopped gradients.

Because $\psi(g) - \psi(s) \approx \phi(s, g)$, you can run the relationship backwards. Take your current state s , take an analogous pair (s_a, g_a) that demonstrates the task you want, and compute $\psi(s) + \phi(s_a, g_a)$. By construction this should land near ψ of the goal you actually want. Since ψ maps individual states, you then look up the nearest neighbor in ψ space among candidate states and read off a concrete goal. No target image required — only an analogy.

$$\psi \left(\text{radishes} \right) + \phi \left(\text{carrots}, \text{diced carrots} \right) = \psi \left(\text{diced radishes} \right)$$

Figure 2. Goals as arithmetic. The relation $\psi(\text{state}) + \phi(\text{analogous task}) = \psi(\text{goal})$ lets the representation of one task — dicing carrots — be applied to a new starting state to produce the corresponding goal, here diced radishes.

The whole system is trained offline. ϕ , ψ , and a policy $\pi(\psi(s), \phi(s, g))$ are learned concurrently on top of Implicit Q-Learning (IQL), from a fixed replay buffer collected by a noisy expert. Both encoders are six-layer CNN networks producing 256-dimensional embeddings.

Does the representation actually behave as intended?

The strongest evidence is qualitative before it is quantitative. If ψ and ϕ really encode functional equivariance, then the arithmetic $\psi(s) + \phi(s_a, g_a)$ should return states that match the analogous task in function while differing in irrelevant detail. The authors compose real state-goal pairs from the simulator and look up the nearest neighbor of the result.

$$\begin{aligned} \psi \left(\text{robot in kitchen} \right) + \phi \left(\text{robot pushing button}, \text{robot pushing button} \right) &= \psi \left(\text{robot pushing button in different pose} \right) \\ \psi \left(\text{robot in kitchen} \right) + \phi \left(\text{robot closing drawer}, \text{robot closing drawer} \right) &= \psi \left(\text{robot closing drawer, ignoring color and pose} \right) \end{aligned}$$

Figure 4. Analogy arithmetic on real states. The rightmost image is the nearest neighbor of $\psi(s) + \phi(s_a, g_a)$. Top: the recovered goal has the finger pushing the button, matching the function of the analogous goal but at a different pose. Bottom: the recovered goal has the drawer closed, matching function while ignoring color and pose.

The recovered goals do the right thing: if the analogy is “press the button,” the arithmetic returns a state with the button pressed, at whatever pose the current scene happens to have; if the analogy is “close the drawer,” it returns a closed drawer, regardless of color. Pose is carried over, color and clutter are ignored — equivariance and invariance in the same picture.

The same structure shows up when you probe ϕ directly. Sampling state-goal pairs and finding their nearest neighbors in ϕ space should surface tasks that are functionally similar but visually different.

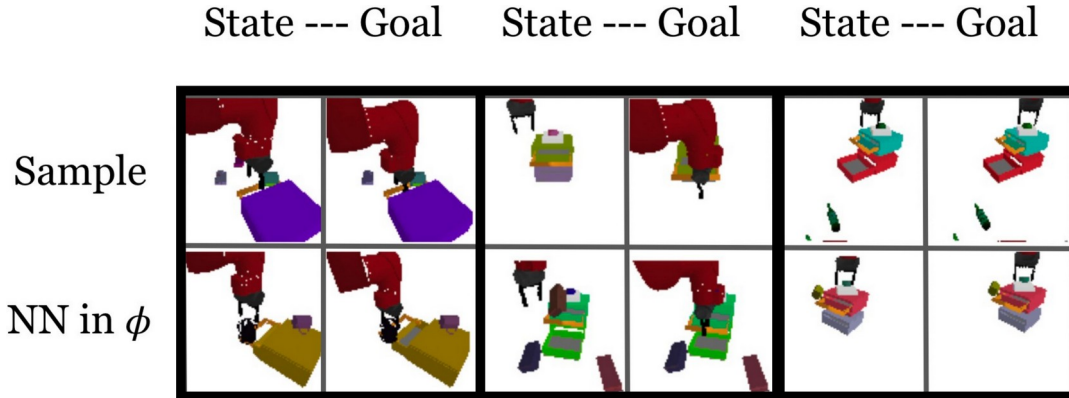


Figure 5. Nearest neighbors in ϕ space. Each sampled state-goal pair (top) sits next to its nearest neighbor (bottom). Task-irrelevant features — drawer color, distractor objects — change freely, while the task-relevant structure — the robot's position relative to the target, the semantics of the action — is preserved.

Neighbors in ϕ share the task, not the appearance — exactly the invariance the vegetable analogy asked for, now made visible directly inside the learned representation space.

Results

Quantitatively, GCB is evaluated in a PyBullet manipulation suite of randomly generated workspaces drawn from 84 object geometries, across three environments — Drawer, Button-and-Drawer (BD), and dedicated Analogy tasks — each also run with Video Distractors (VD) overlaid in the background. Success is measured against the true goal over 5 seeds, against eight representation-learning baselines.

Table 1. Final average success rate over 5 seeds (standard errors reported in the paper). BD = Button and Drawer; VD = Video Distractors. GCB is best on 5 of 6 settings and strongest on every distractor-augmented task; on Analogy it is the only method that infers the goal from an example, roughly doubling the next-best baseline.

Method	Drawer	Drawer+VD	BD	BD+VD	Analogy	Analogy+VD
GCB (ours)	0.533	0.448	0.440	0.322	0.403	0.303
CPV	0.400	0.209	0.228	0.198	0.176	0.121
GC-DBC	0.269	0.261	0.358	0.255	NA	NA
CCVAE	0.443	0.231	0.452	0.225	0.234	0.095
VAE	0.463	0.261	0.522	0.115	0.280	0.112
CPC	0.305	0.207	0.363	0.154	0.148	0.145
CURL	0.259	0.283	0.382	0.198	0.162	0.123
RAD	0.245	0.207	0.159	0.108	0.157	0.110
Pixel	0.483	0.256	0.117	0.136	0.132	0.121

Two patterns stand out. First, GCB wins on every task with video distractors — 0.448 on Drawer+VD, 0.322 on BD+VD, 0.303 on Analogy+VD — which is what you would expect from a representation built to discard task-irrelevant variation. Second, on the Analogy tasks, where the goal must be inferred from an example rather than handed over, GCB reaches 0.403 against 0.176 for CPV, the next-best method — a near-doubling. The one setting it does not win is plain BD without distractors, where a VAE baseline (0.522) edges ahead; distractor-free, low-variation tasks are where a generic reconstruction objective is hardest to beat.

Ablations back the main design choices. Adding a grounding term $\phi(g_i, g_i)$ as a normalizing constant in ψ 's objective helps; fitting ϕ with an ℓ_1 metric loss beats ℓ_2 ; and varying the latent dimensionality from 32 to 512 barely moves downstream success, so 256 is a safe default rather than a delicate tuning.

Why it matters

The conceptual shift is small to state and large in consequence: treat bisimulation as an equivalence over tasks rather than over states. Once behaviorally equivalent state-goal pairs map to the same representation, goals become composable, and an agent can be pointed at an example instead of a target image. The authors also prove the learned representation is sufficient not just for goal-conditioned control but for any task defined by a state-only reward function.

The honest boundary is scope: the results are in simulation, on manipulation tasks with a demonstration policy and an offline buffer, and the analogy still has to come from the same task family. But as an interface, “show me a similar task” is far closer to how people actually delegate work than “give me the exact final image” — and this paper is a concrete step toward representations that support it.